Linux内核如何装载和启动一个可执行程序

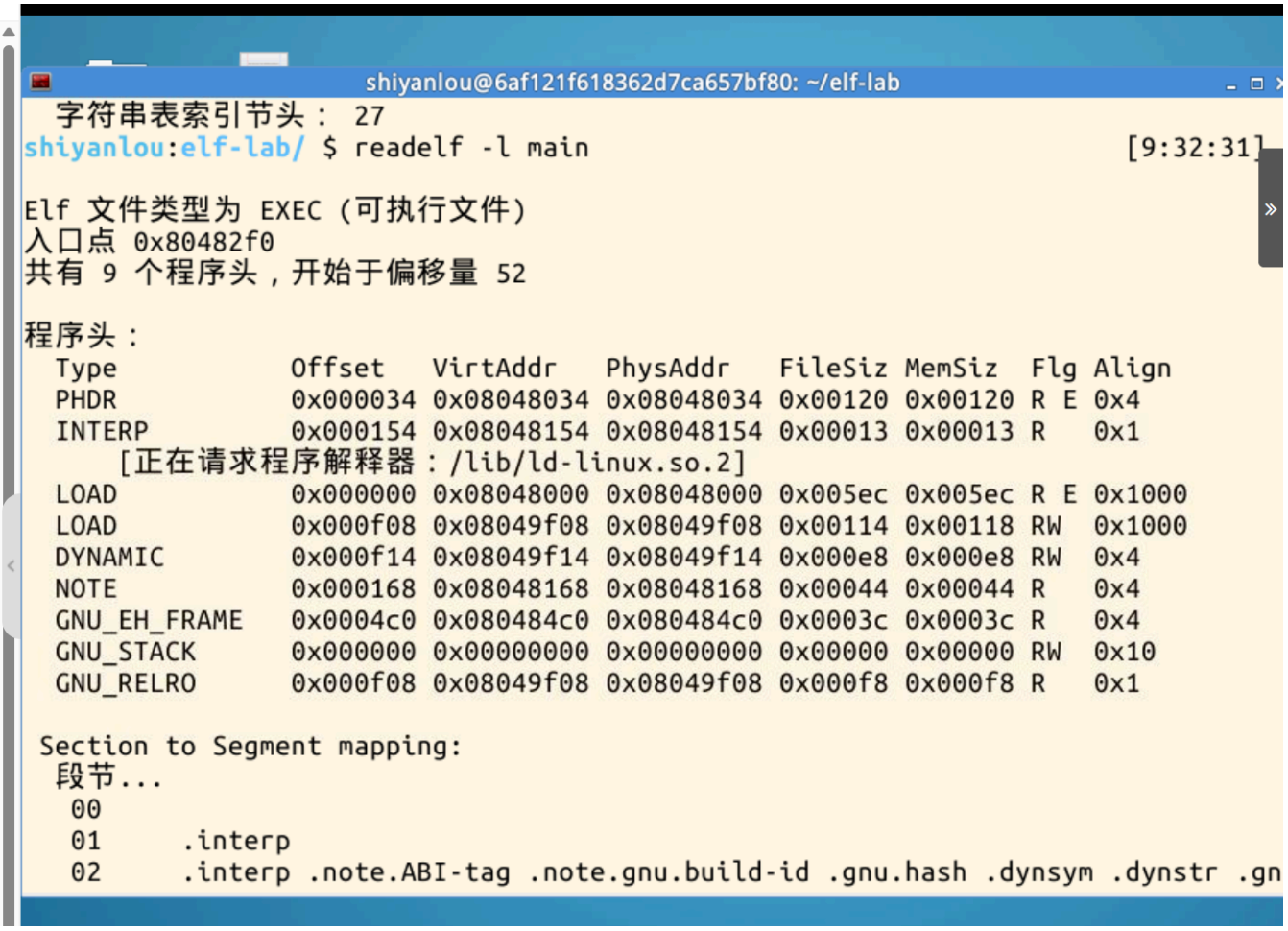
一、编译、链接与ELF

沿用第二周作业的`main.c`。用`gcc -m32 -S`生成汇编`main.s`，再用`gcc -m32 -c`生成目标文件`main.o`，最后链接成`main`。两个二进制文件都是`ELF32`。`main.o`的类型是`REL`，程序头数量为0。`main`的类型是`EXEC`，有9个程序头，入口地址`0x080482f0`对应`_start`。

```
.LFB1:
shiyanlou:elf-lab/ $ gcc -m32 -c main.s -o main.o && readelf -h main.o
ELF 头：
  Magic：   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              REL (可重定位文件)
  Machine:                           Intel 80386
  Version:                           0x1
  入口点地址：               0x0
  程序头起点：          0 (bytes into file)
  Start of section headers:          352 (bytes into file)
  标志：             0x0
  本头的大小：       52 (字节)
  程序头大小：       0 (字节)
  Number of program headers:         0
  节头大小：         40 (字节)
  节头数量：         12
  字符串表索引节头： 9
shiyanlou:elf-lab/ $                                              [9:32:02]
```

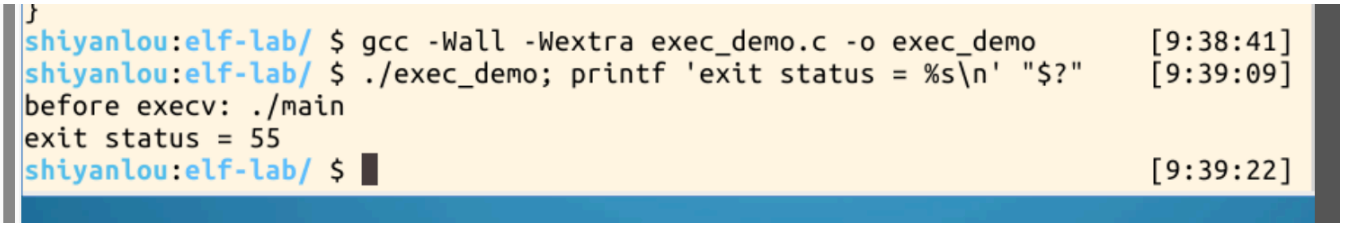
```
  字符串表索引节头： 9
shiyanlou:elf-lab/ $ gcc -m32 main.o -o main && readelf -h main       [9:32:02]
ELF 头：
  Magic：   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (可执行文件)
  Machine:                           Intel 80386
  Version:                           0x1
  入口点地址：               0x80482f0
  程序头起点：          52 (bytes into file)
  Start of section headers:          4424 (bytes into file)
  标志：             0x0
  本头的大小：       52 (字节)
  程序头大小：       32 (字节)
  Number of program headers:         9
  节头大小：         40 (字节)
  节头数量：         30
  字符串表索引节头： 27
shiyanlou:elf-lab/ $                                              [9:32:31]
```

`readelf -l` 显示，两个 `LOAD` 段分别具有可读可执行和可读可写权限。`INTERP` 指定动态加载器 `/lib/ld-linux.so.2`。



二、execv与两种动态链接

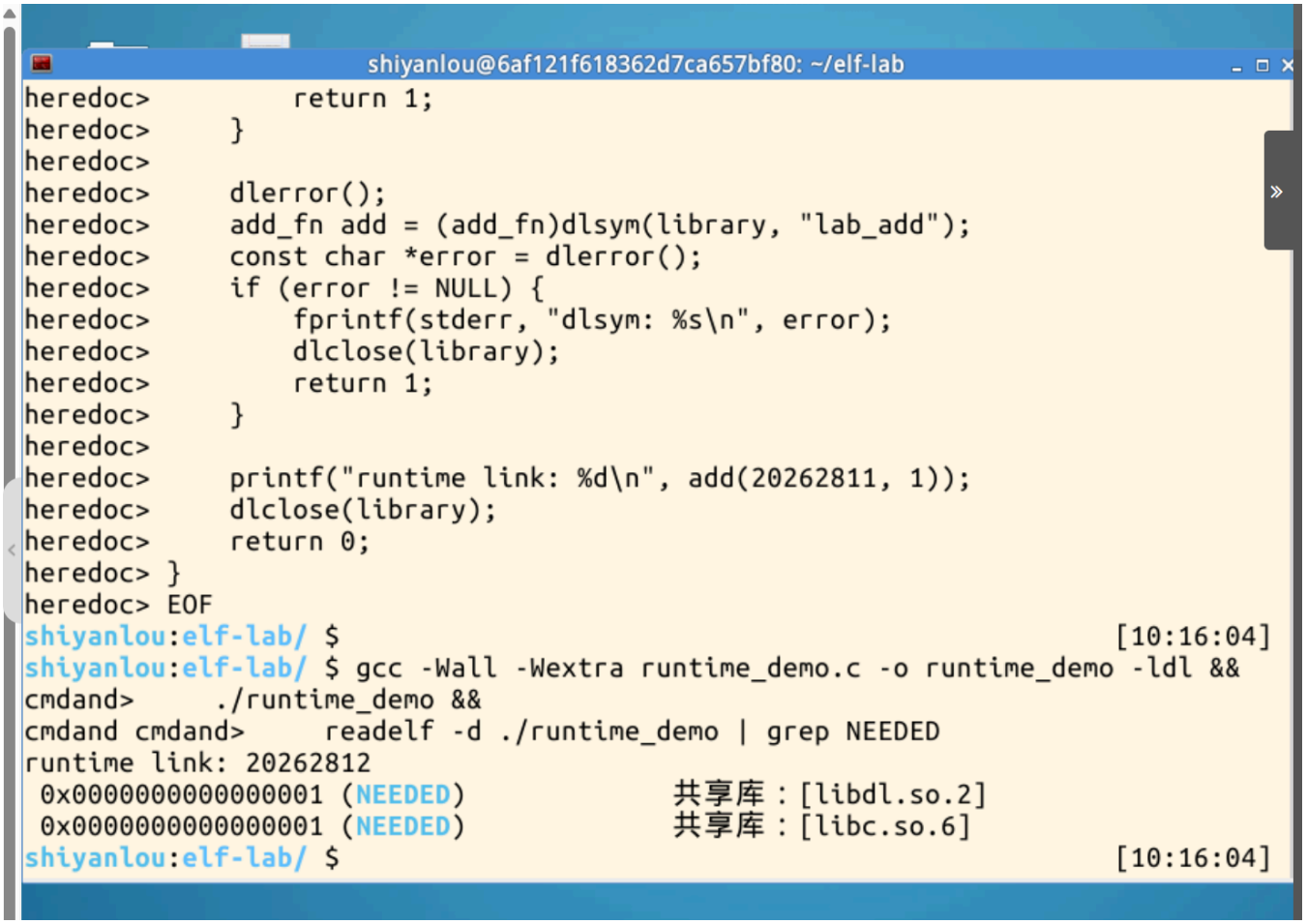
`exec_demo` 调用 `execv()` 装载 `./main`。Shell读到退出状态 `55`，与直接运行 `./main` 一致。



装载时链接的 `startup_demo` 把 `liblabmath.so` 写入 `NEEDED` 依赖表。启动时装载该库，调用 `lab_add(20262811, 1)` 得到 `20262812`。

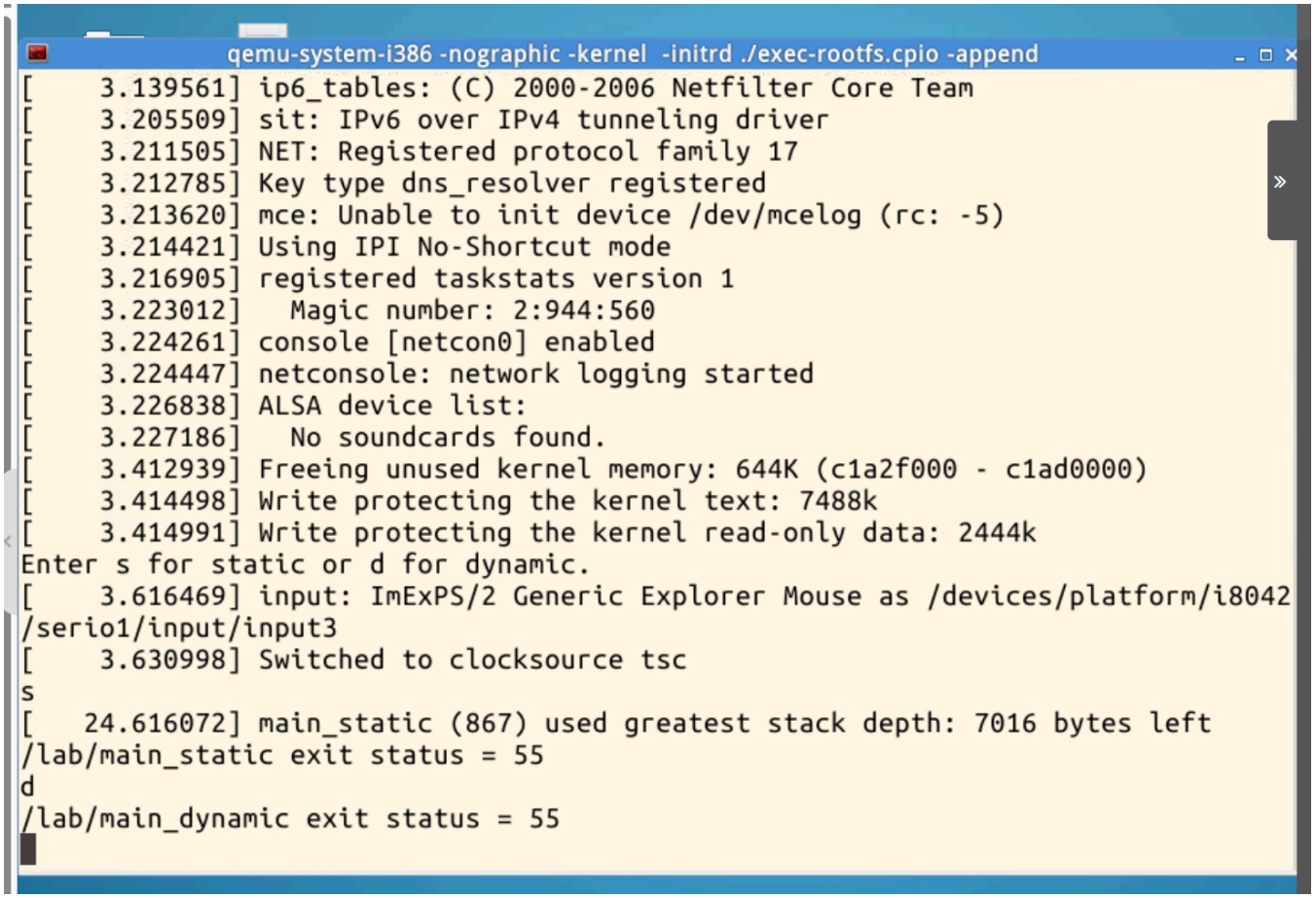


运行时链接的 `runtime_demo` 进入 `main()` 后才用 `dlopen()` 打开同一个库，再用 `dlsym()` 找到 `lab_add()`。输出也是 `20262812`，而 `NEEDED` 只列出 `libdl.so.2` 和 `libc.so.6`。



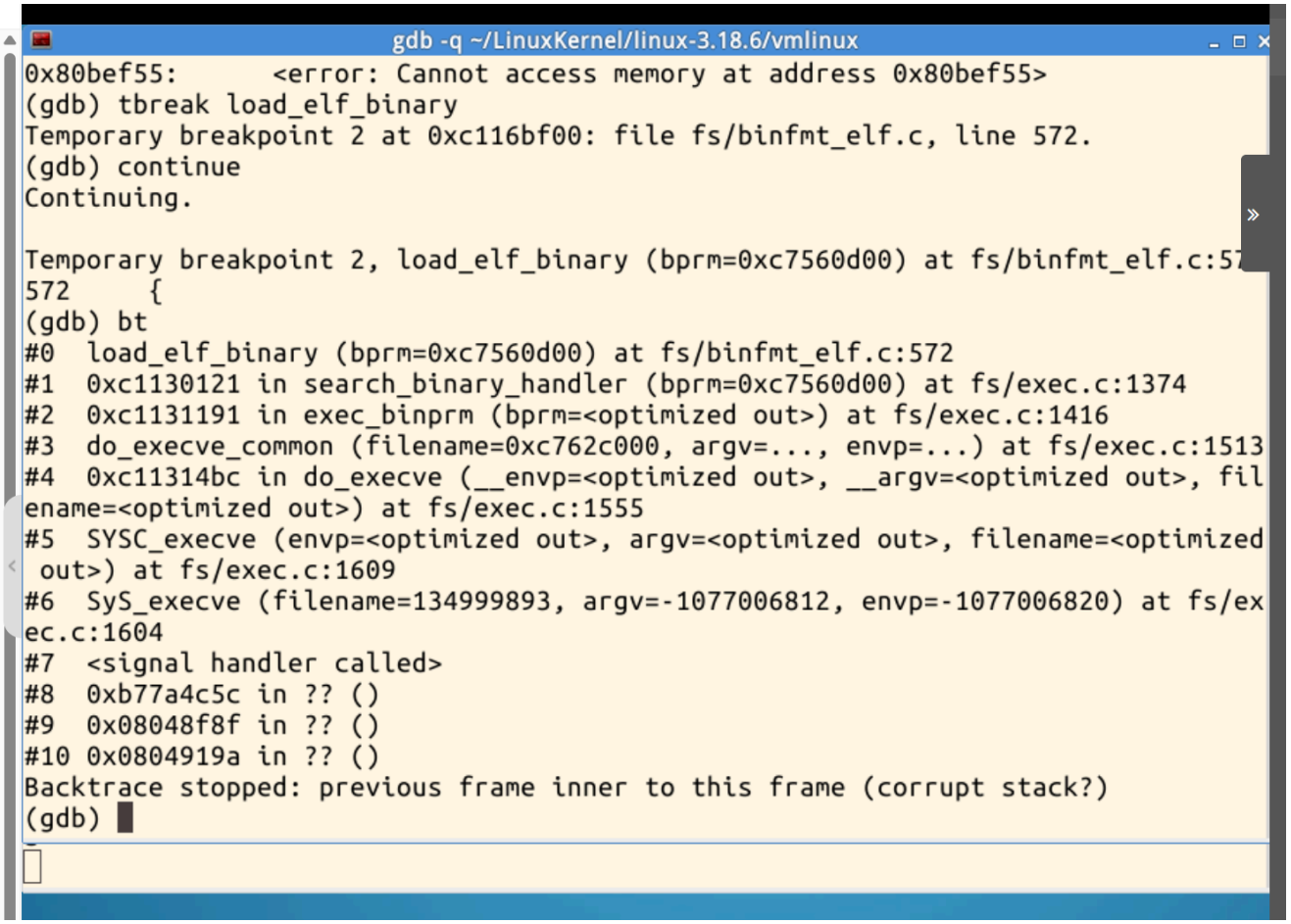
三、用GDB跟踪execve

用同一个`main.o`生成静态版和动态版。QEMU里的启动器让子进程调用`execve()`，两版的退出状态都是`55`。



GDB从`sys_execve`跟到`load_elf_binary()`。入口处直接读取用户地址时出现报错，随后在ELF装载函数中从`bprm->filename`读到了目标路径。

```
sys_execve → SYSC_execve → do_execve → do_execve_common
  → exec_binprm → search_binary_handler → load_elf_binary
```



在两版各自的第一条用户态指令处停住，得到以下实测值。

链接方式	`EIP` **及起点**	**初始** `ESP`	`argv[0]`
静态	`0x08048d0a`，主程序入口	`0xbf83bd30`	`/lab/main_static`
动态	`0xb77c30d0`，加载器入口	`0xbfb46270`	`/lab/main_dynamic`

两版的初始栈都是 `argc = 2`，`argv[1]` 都是 `20262811`。参数表和环境变量表分别以空指针结束，之后是辅助向量。

```
(gdb) thbreak *0x08048d0a
Hardware assisted breakpoint 4 at 0x8048d0a
(gdb) continue
Continuing.

Temporary breakpoint 4, 0x08048d0a in ?? ()
(gdb) info registers eip esp cs
eip            0x8048d0a        0x8048d0a
esp            0xbf83bd30       0xbf83bd30
cs             0x73     115
(gdb) x/6wx $esp
0xbf83bd30:     0x00000002      0xbf83cfba      0xbf83cfcb      0x00000000
0xbf83bd40:     0xbf83cfd4      0x00000000
(gdb) x/s *(char **)($esp + 4)
0xbf83cfba:     "/lab/main_static"
(gdb) x/s *(char **)($esp + 8)
0xbf83cfcb:     "20262811"
(gdb)
```

动态版的`AT_BASE`为`0xb77c2000`，加上加载器ELF头里的入口偏移`0x10d0`，正好是实测的`EIP`。`AT_ENTRY = 0x080482f0`则给出了主程序的入口。

```
Temporary breakpoint 7, 0xb77c30d0 in ?? ()
(gdb) info registers eip esp cs
eip            0xb77c30d0       0xb77c30d0
esp            0xbfb46270       0xbfb46270
cs             0x73     115
(gdb) x/32wx $esp
0xbfb46270:     0x00000002      0xbfb46fb8      0xbfb46fca      0x00000000
0xbfb46280:     0xbfb46fd3      0x00000000      0x00000020      0xb77c1c4c
0xbfb46290:     0x00000021      0xb77c1000      0x00000010      0x0780abf9
0xbfb462a0:     0x00000006      0x00001000      0x00000011      0x00000064
0xbfb462b0:     0x00000003      0x08048034      0x00000004      0x00000020
0xbfb462c0:     0x00000005      0x00000009      0x00000007      0xb77c2000
0xbfb462d0:     0x00000008      0x00000000      0x00000009      0x080482f0
0xbfb462e0:     0x0000000b      0x00000000      0x0000000c      0x00000000
(gdb) x/s *(char **)($esp + 4)
0xbfb46fb8:     "/lab/main_dynamic"
(gdb) x/s *(char **)($esp + 8)
0xbfb46fca:     "20262811"
(gdb) shell LC_ALL=C readelf -h ~/elf-lab/exec-root/lib/ld-linux.so.2 | grep 'En
try point address'
  Entry point address:               0x10d0
(gdb)
```

小结

ELF程序头描述文件怎样映射到内存。装载完成后，`start_thread()`把新入口和栈指针写入保存的用户态寄存器。`execve()`成功后，系统调用退出路径恢复的是这个新现场，所以CPU会从新映像开始执行。静态版直接进入主程序，动态版先由加载器完成启动所需的链接，再把控制权交给主程序的`_start`。[Linux 3.18的x86入口设置代码]